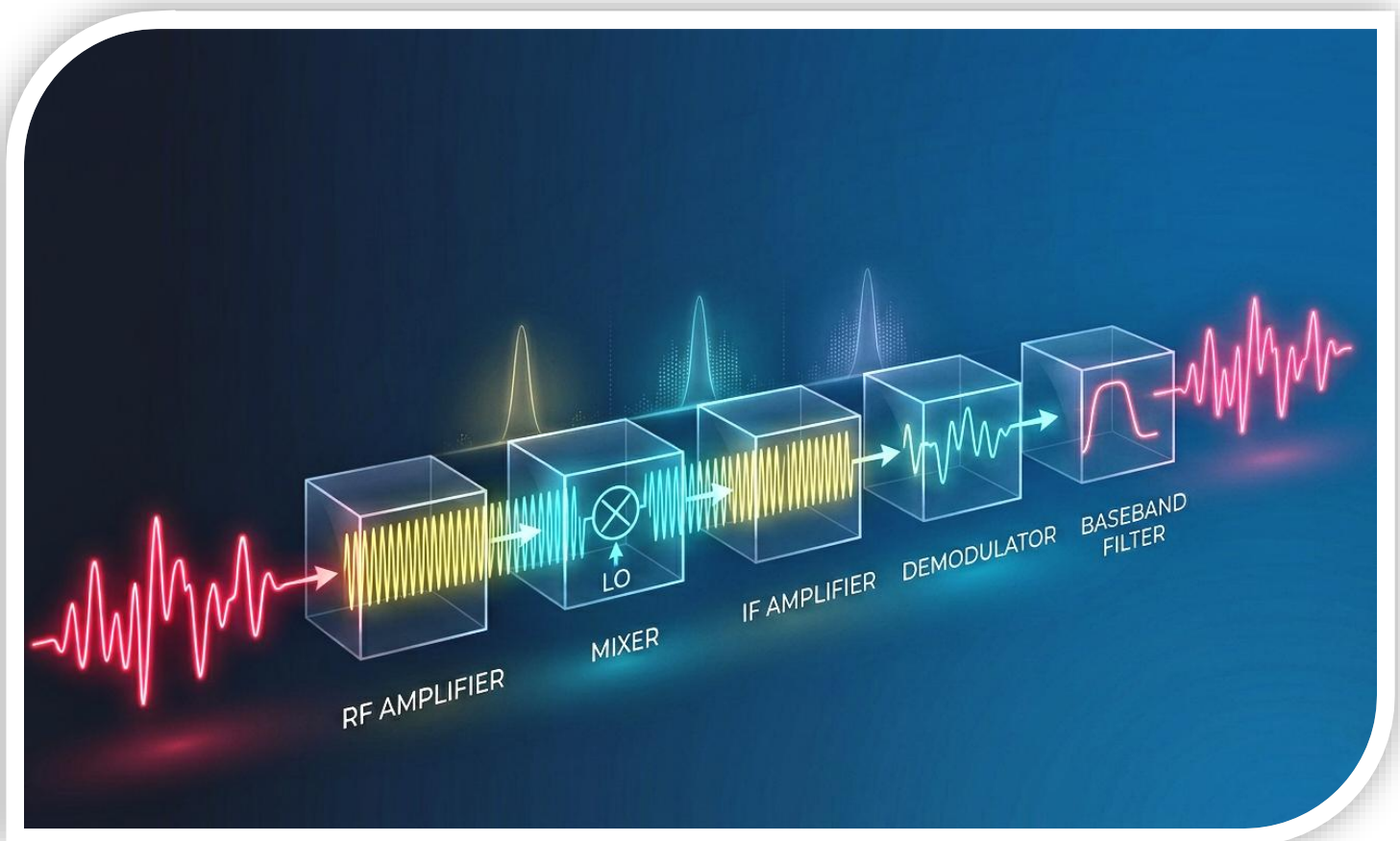




Electronics and Electrical Communications Department
Second Year – Second Semester
Introduction to Communication Systems - EECG 232



Super-heterodyne Receiver & DSB-SC AM Communication System



Under Supervision of: **Eng. Mohammed Khaled**

Mohaned Waleed Hosny

91241081

Mostafa Abdelhamid Muhammad

91240761

1. Introduction

The objective of this project is to simulate the fundamental components of an analog communication system using MATLAB programming. Specifically, the project involves implementing a Double Sideband Suppressed Carrier (DSB-SC) AM modulator and a corresponding super-heterodyne receiver to process, multiplex, and accurately recover multiple radio-station-generated audio signals.

This simulation highlights the complete transmission and reception chain, including signal preprocessing, upsampling, AM modulation, Frequency Division Multiplexing (FDM), RF front-end filtering, mixing with a local oscillator, intermediate frequency (IF) processing, and baseband detection. It also explores critical experimental scenarios such as the absence of RF filtering and frequency offsets in the local oscillator, illustrating the practical effects on signal quality, spectrum, and intelligibility.

2. System Blocks

2.1 Signal Preprocessing

Explanation: The provided stereo audio signals are converted into monophonic channels by summing the two original channels. To ensure consistent matrix processing, the signals are zero-padded so that all signals share the same length, despite differing durations. This step prepares the data for uniform subsequent processing such as interpolation, modulation, and multiplexing.

MATLAB Code Snippet:

```
%% 1. Signal Preprocessing
disp('--- Stage 1: Preprocessing ---');
fileNames = {'audio1.wav', 'audio2.wav', 'audio3.wav', 'audio4.wav', 'audio5.wav'};
numSig = length(fileNames); audioSignals = cell(1, numSig);
fs_orig = zeros(1, numSig); maxLength = 0;

for i = 1:numSig
    [raw, fs] = audioread(fileNames{i});
    fs_orig(i) = fs; mono = sum(raw, 2);
    mono = mono / max(abs(mono)); audioSignals{i} = mono;
    maxLength = max(maxLength, length(mono));
end

paddedSignals = zeros(maxLength, numSig);
for i = 1:numSig, paddedSignals(1:length(audioSignals{i}), i) = audioSignals{i}; end
```

Note: This code outputs the baseband spectrum figure via the later plotSpectrum calls.

2.2 Interpolation & Bandwidth Estimation

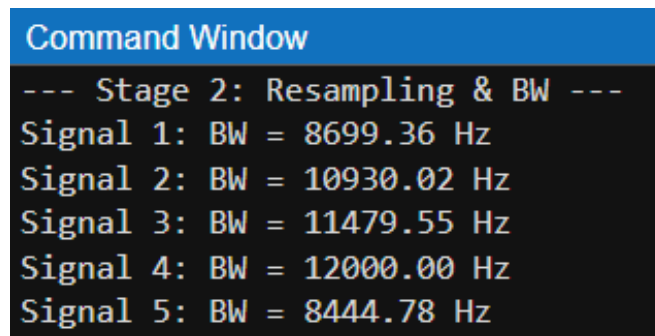
Explanation: To avoid aliasing when modulating onto high-frequency carriers, each baseband signal is upsampled using **interp** to increase the sampling rate. The bandwidth of each signal is estimated using FFT and `estimateBandwidth`, which finds the frequency range where the signal has significant energy (>1% of peak). This bandwidth is later used to design the RF and IF filters.

MATLAB Code Snippet:

```
%% 2. Interpolation & BW Estimation
disp('--- Stage 2: Resampling & BW ---');
baseFs = fs_orig(1); interpFactor = ceil(600000 / baseFs);
newFs = baseFs * interpFactor; interpLen = maxLength * interpFactor;
processedSignals = zeros(interpLen, numSig); estBW = zeros(1, numSig);

for i = 1:numSig
    estBW(i) = min(estimateBandwidth(paddedSignals(:,i), baseFs), 12000);
    fprintf('Signal %d: BW = %.2f Hz\n', i, estBW(i));
    processedSignals(:,i) = interp(paddedSignals(:,i), interpFactor);
end
```

Note: The output includes estimated bandwidth values used for filter design.



The image shows a screenshot of a MATLAB Command Window. The title bar is blue and says "Command Window". The background is black with white text. The output shows a separator line "--- Stage 2: Resampling & BW ---" followed by five lines of signal bandwidth estimates. Each line is "Signal X: BW = Y Hz" where X is the signal number and Y is the estimated bandwidth in Hz.

Signal	Estimated Bandwidth (Hz)
Signal 1	8699.36
Signal 2	10930.02
Signal 3	11479.55
Signal 4	12000.00
Signal 5	8444.78

Figure 1: Estimated bandwidths of all audio signals in the command window.

2.3 AM Modulation (DSB-SC)

Explanation: Each upsampled audio signal is modulated using a DSB-SC scheme by multiplying it with a cosine carrier of frequency $f_c(i)$. The carriers are spaced 30 kHz apart starting from 100 kHz. The `plotSpectrum` function is used to visualize both the original baseband signals and their modulated RF spectra.

MATLAB Code Snippet:

```
%% 3. AM Modulation & Comparative Spectra
disp('--- Stage 3: AM Modulation ---');
t = (0:interpLen-1) / newFs; modulatedSignals = zeros(interpLen, numSig);
fc = 100000 + (0:numSig-1)*30000;
figure('Name','Stage 3: Comparative Spectra'); colors = lines(numSig);

for i = 1:numSig
    modulatedSignals(:,i) = processedSignals(:,i) .* cos(2*pi*fc(i)*t);
    subplot(2,1,1); hold on; plotSpectrum(processedSignals(:,i), newFs, "r", colors(i,:));
    subplot(2,1,2); hold on; plotSpectrum(modulatedSignals(:,i), newFs, "r", colors(i,:));
end
subplot(2,1,1); title('All Baseband Spectra'); grid on; legend show; xlim([-max(estBW)*1.2, max(estBW)*1.2]);
subplot(2,1,2); title('All Modulated Spectra'); grid on; legend show; xlim([-max(fc)+max(estBW),
max(fc)+max(estBW)]);
```

Note: This generates the Comparative Spectra figure showing the shift from baseband to RF.

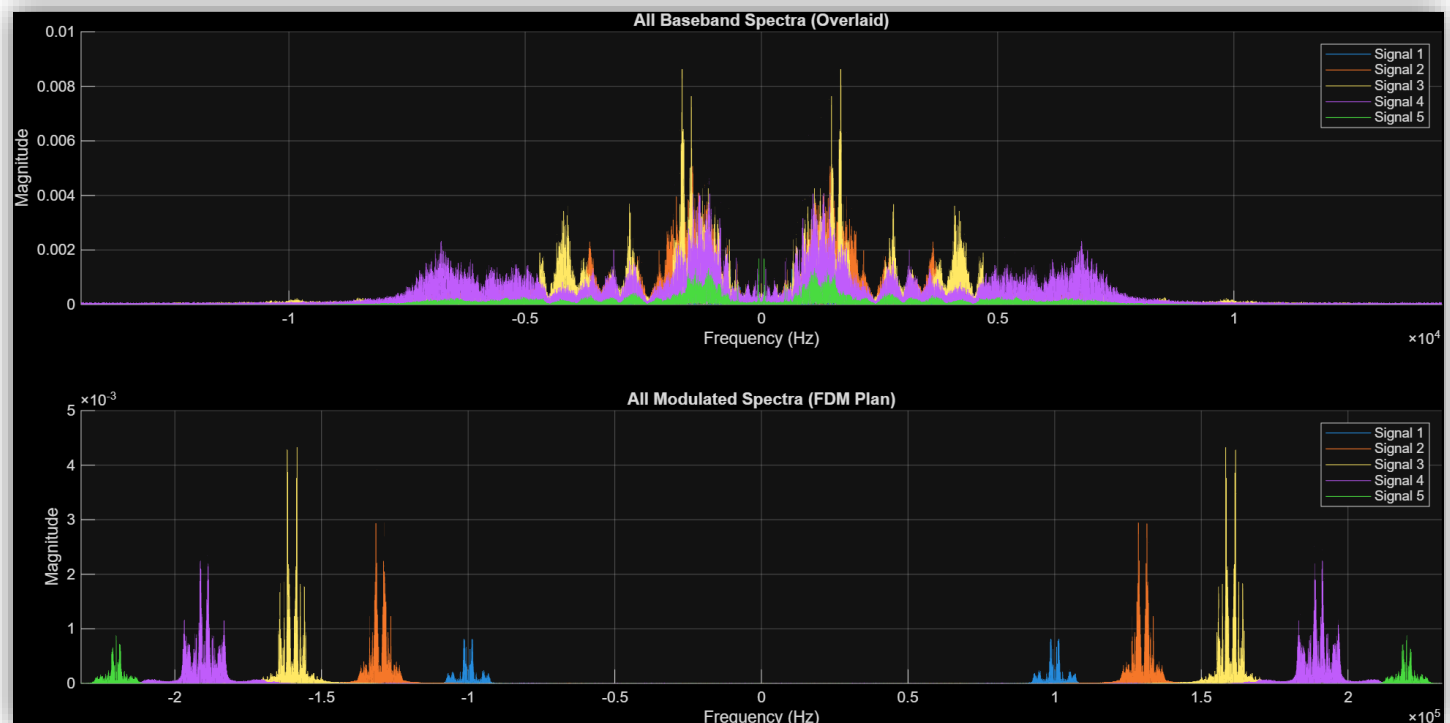


Figure 2: Baseband vs. AM Modulated Spectra.

2.4 Frequency Division Multiplexing (FDM)

Explanation: All modulated signals are summed to produce a single FDM signal. This simulates transmitting multiple stations simultaneously on different frequency slots. The spectrum of the FDM signal is plotted to show all channels occupying their respective carrier frequencies.

MATLAB Code Snippet:

```
%% 4. Frequency Division Multiplexing (FDM)
disp('--- Stage 4: FDM ---');
fdmSignal = sum(modulatedSignals, 2);
figure('Name','Stage 4: FDM'); plotSpectrum(fdmSignal, newFs, 'FDM Spectrum', 'b');
xlim([-max(fc)+max(estBW), max(fc)+max(estBW)]);
```

Note: This generates the FDM spectrum figure showing all channels occupying their respective frequency slots.

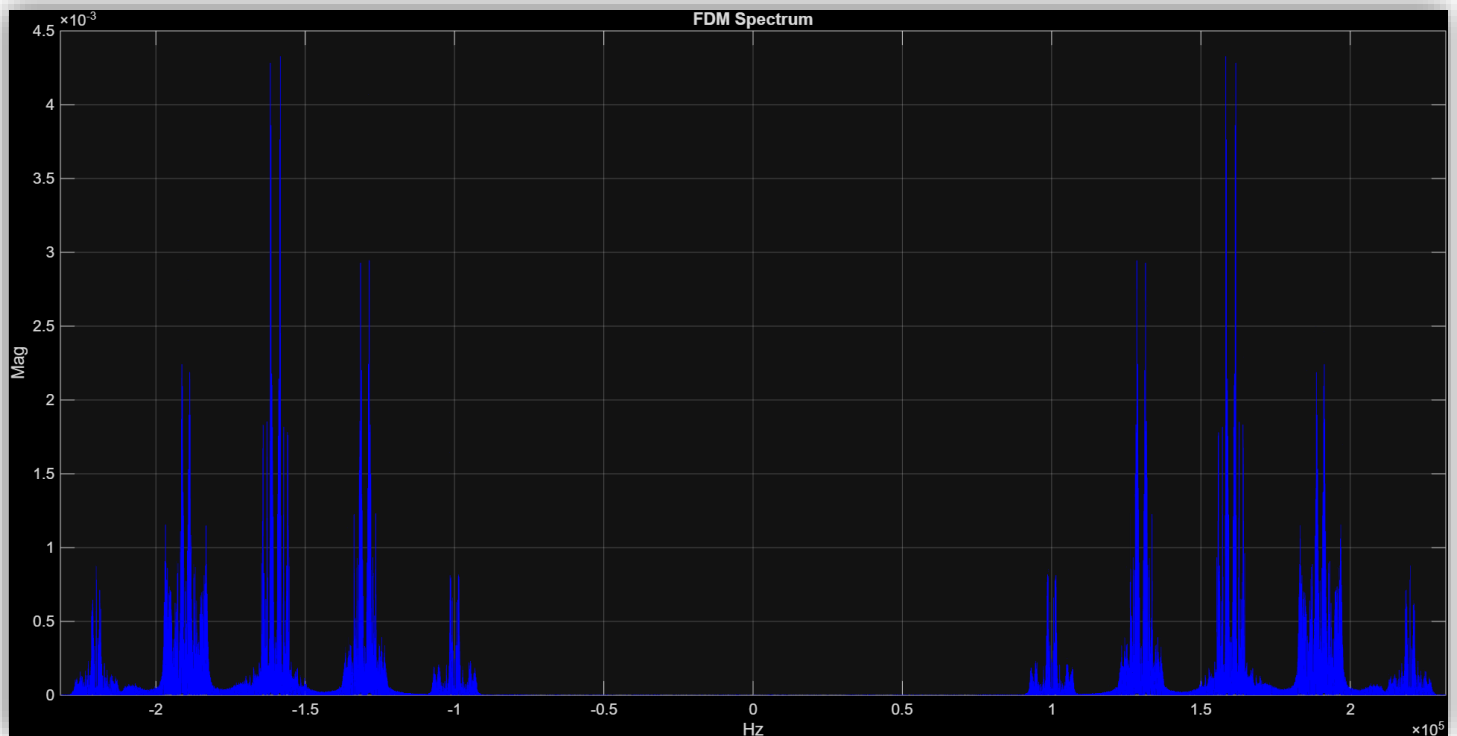


Figure 3: FDM Signal Spectrum.

2.5 RF Stage

Explanation: The RF stage models the receiver front-end by applying a band-pass filter (**designfilt**) centered at the desired station's carrier (**targetFc**). This isolates the target signal from the other stations and performs image-frequency rejection. The filtered RF signal is visualized using **plotSpectrum**.

MATLAB Code Snippet:

```
%% 5. Receiver - RF Stage
disp('--- Stage 5: RF Stage ---');
targetIdx = 3; targetFc = fc(targetIdx); bw = estBW(targetIdx);
rfFilt = designfilt('bandpassiir','FilterOrder',4,'HalfPowerFrequency1',targetFc-bw,...
    'HalfPowerFrequency2',targetFc+bw,'SampleRate',newFs);
rfOut = filtfilt(rfFilt, fdmSignal);
figure('Name','Stage 5: RF Output'); plotSpectrum(rfOut, newFs, 'RF Output', 'b');
xlim([-targetFc + 2*bw, targetFc + 2*bw]);
```

Note: This generates the RF output spectrum figure isolating the targeted station.

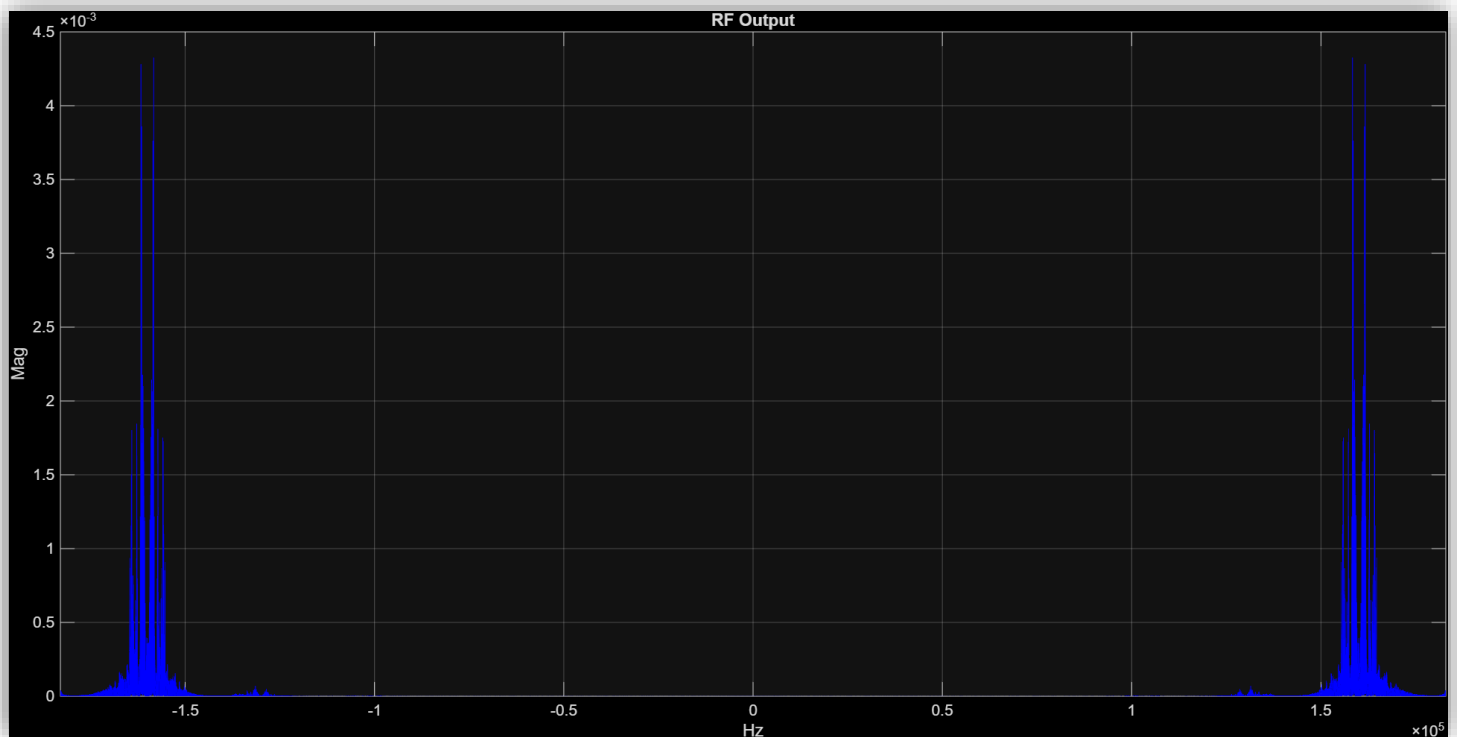


Figure 4: Isolated RF Output Spectrum.

2.6 Mixer & Local Oscillator

Explanation: The RF output is mixed with a local oscillator ($f_{LO} = \text{target}F_c + f_{IF}$) to shift the signal down to the intermediate frequency (IF). Multiplication in the time domain corresponds to a frequency shift in the spectrum, which is visualized to show the down-converted signal.

MATLAB Code Snippet:

```
%% 6. Mixer & Local Oscillator
disp('--- Stage 6: Mixer ---');
fIF = 15000; fLO = targetFc + fIF;
mixedSignal = rfOut .* cos(2*pi*fLO*t);
figure('Name','Stage 6: Mixer'); plotSpectrum(mixedSignal, newFs, 'Mixer Output', 'b');
xlim([-fLO + targetFc + bw, (fLO + targetFc + bw)]);
```

Note: This generates the Mixer output spectrum figure.

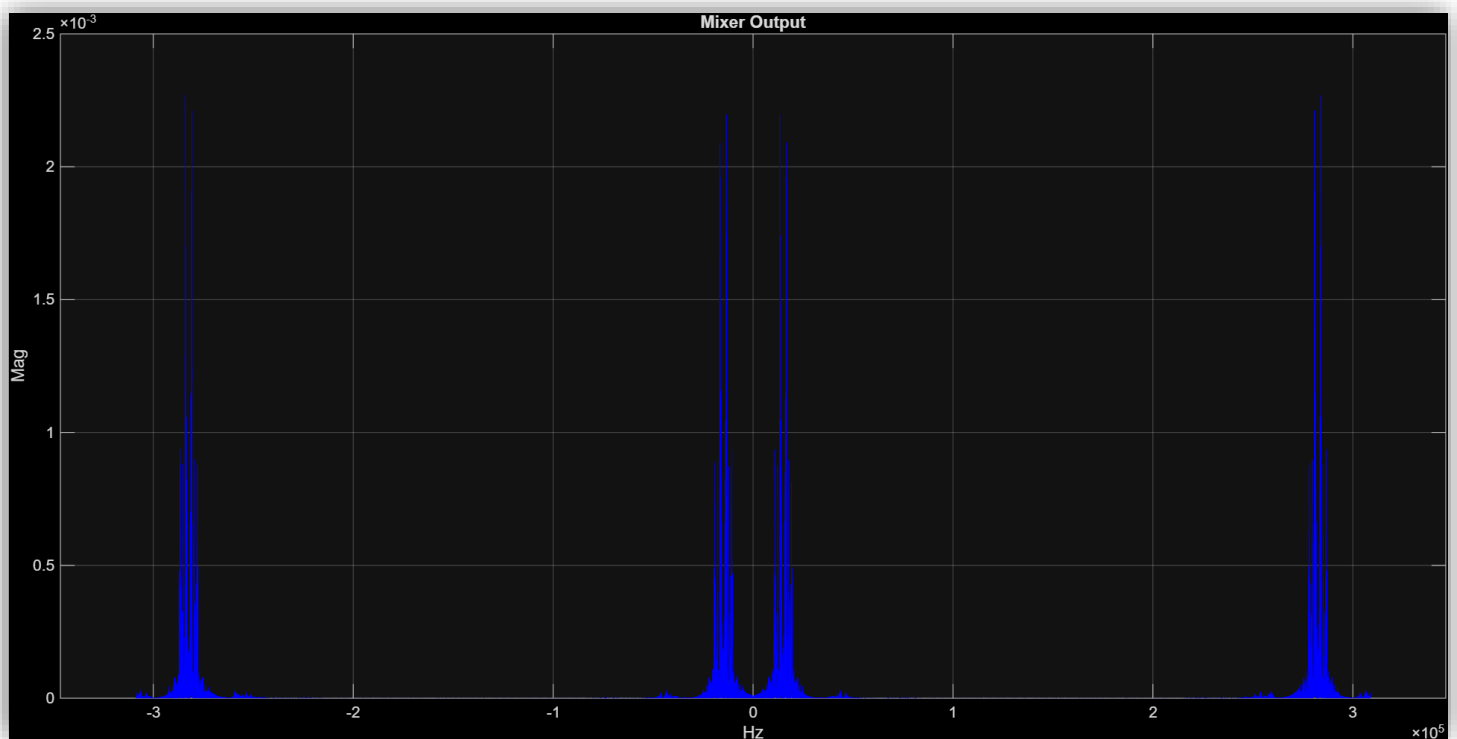


Figure 5: Mixer Output Spectrum.

2.7 IF Stage

Explanation: The IF stage applies a highly selective band-pass filter centered at f_{IF} to remove unwanted spectral components after mixing. This ensures only the desired down-converted signal is passed to the baseband detection stage.

MATLAB Code Snippet:

```
%% 7. IF Stage
disp('--- Stage 7: IF Stage ---');
ifFilt = designfilt('bandpassiir','FilterOrder',4,'HalfPowerFrequency1',fIF-bw,...
    'HalfPowerFrequency2',fIF+bw,'SampleRate',newFs);
ifOut = filtfilt(ifFilt, mixedSignal);
figure('Name','Stage 7: IF Output'); plotSpectrum(ifOut, newFs, 'IF Output', 'b');
xlim([-fIF + 2*bw, fIF + 2*bw]);
```

Note: This generates the IF output spectrum figure.

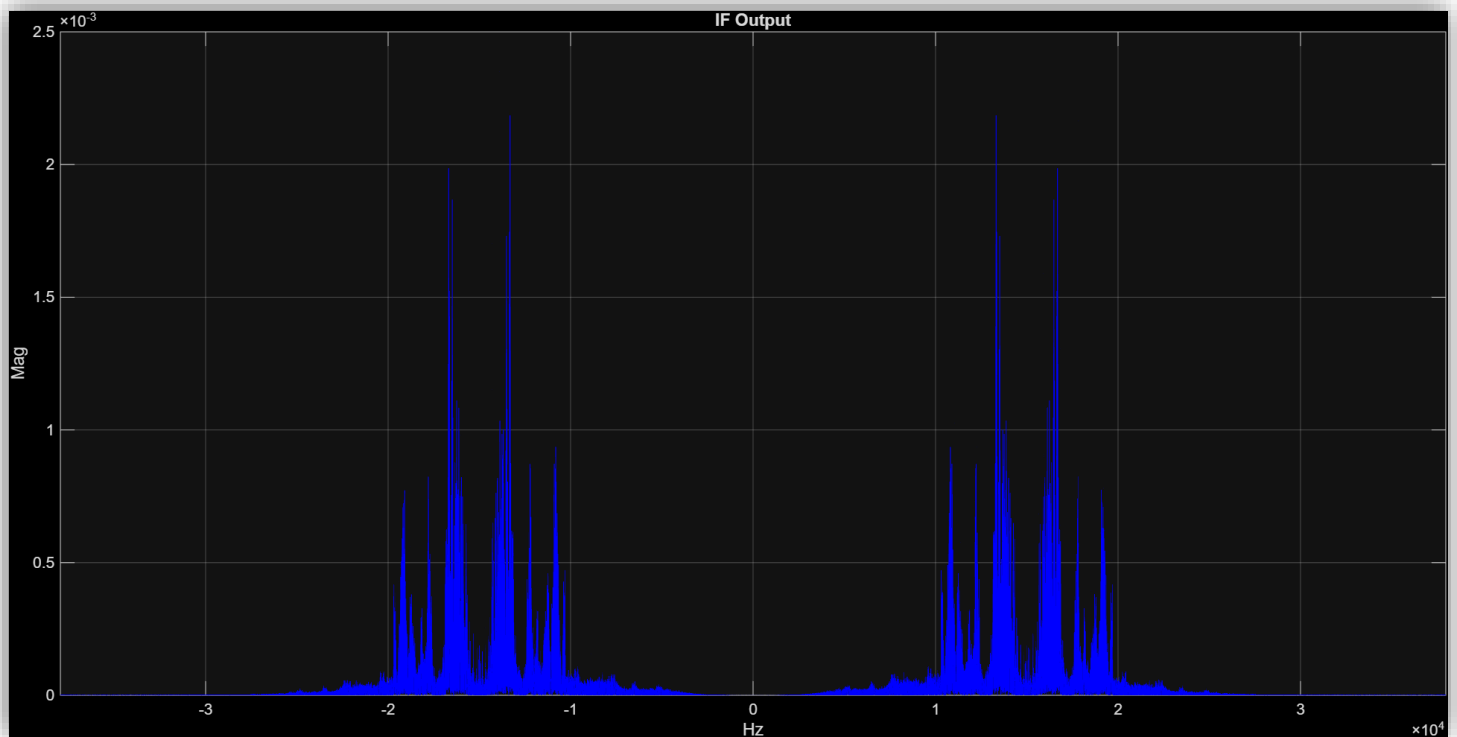


Figure 6: Filtered IF Output Spectrum.

2.8 Baseband Detection

Explanation: The IF signal is multiplied by a cosine at f_{IF} and filtered with a low-pass filter to recover the original audio message. The recovered signal is normalized, visualized in the frequency domain, and played back using sound.

MATLAB Code Snippet:

```
%% 8. Baseband Detection & Playback
disp('--- Stage 8: Demodulation ---');
detMixed = ifOut .* cos(2*pi*fIF*t);
lpFilt = designfilt('lowpassiir','FilterOrder',6,'HalfPowerFrequency',bw,'SampleRate',newFs);
recovered = filtfilt(lpFilt, detMixed); recovered = recovered / max(abs(recovered));

figure('Name','Stage 8: Recovered'); plotSpectrum(recovered, newFs, 'Recovered Spectrum', 'b');
xlim([-bw*1.2, bw*1.2]);
disp('Playing recovered audio...'); sound(downsample(recovered, interpFactor), baseFs);
pause(maxLength/baseFs + 1);
```

Note: This generates the final recovered audio spectrum figure.

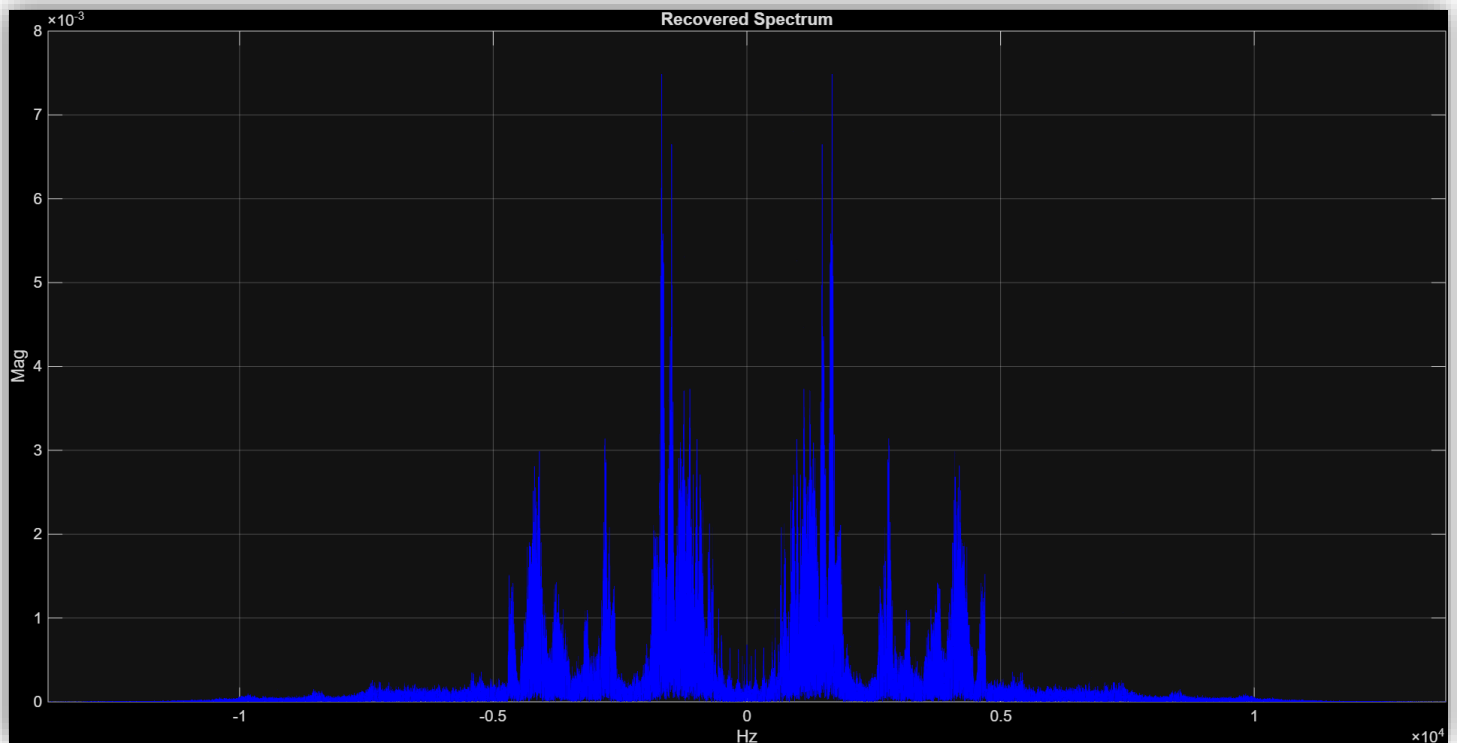


Figure 7: Recovered Baseband Audio Spectrum.

2.9 Experiments

Exp A: No RF Filter

Explanation: By bypassing the RF filter, the FDM signal is directly mixed with the LO. This allows image frequencies from other stations to interfere, which is visible in the IF spectrum and audible as distorted audio.

MATLAB Code Snippet:

```
% Exp A: No RF Filter
disp('Exp A: No RF Filter (Playing audio...)');
mixedNoRF = fdmSignal .* cos(2*pi*fLO*t);
ifOutNoRF = filtfilt(ifFilt, mixedNoRF);
recNoRF = filtfilt(lpFilt, ifOutNoRF .* cos(2*pi*fIF*t));

figure('Name','Experiment A: Effect of Removing RF Filter');
subplot(3,1,1); plotSpectrum(fdmSignal, newFs, 'Input to Mixer (Full FDM - No RF Filter)', 'r');
xlim([-max(fc)+max(estBW), max(fc)+max(estBW)]);
subplot(3,1,2); plotSpectrum(ifOutNoRF, newFs, 'IF Output (Image Frequencies Present)', 'r');
xlim([-fIF + 2*bw, fIF + 2*bw]);
subplot(3,1,3); plotSpectrum(recNoRF, newFs, 'Recovered Baseband (Distorted)', 'r');
xlim([-bw*1.2, bw*1.2]);

sound(downsample(recNoRF/max(abs(recNoRF)), interpFactor), baseFs); pause(maxLength/baseFs + 1);
```

Note: Generates the IF output spectrum figure demonstrating interference.

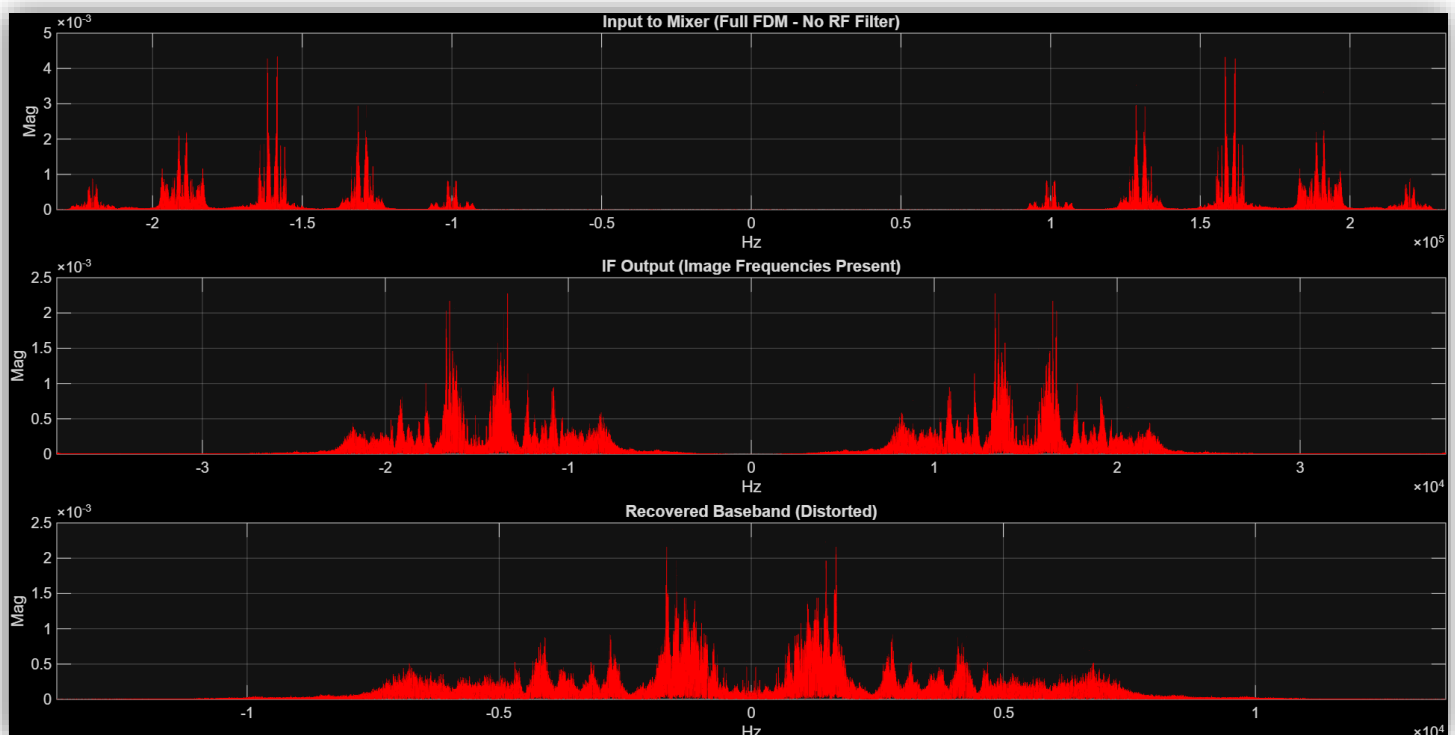


Figure 8: Effect of removing RF filter.

Exp B: LO Frequency Offset

Explanation: Shifting the LO frequency by 0.1 kHz and 1 kHz simulates oscillator inaccuracies. The resulting spectra are slightly or severely misaligned, causing pitch changes and audio distortion. The recovered signals are visualized and played to demonstrate the effect.

MATLAB Code Snippet:

```
% Exp B: LO Offset
off1 = 100; off2 = 1000;
recOff1 = filtfilt(lpFilt, filtfilt(ifFilt, rfOut .* cos(2*pi*(fLO+off1)*t)) .* cos(2*pi*fIF*t));
recOff2 = filtfilt(lpFilt, filtfilt(ifFilt, rfOut .* cos(2*pi*(fLO+off2)*t)) .* cos(2*pi*fIF*t));

disp('Exp B: LO Offset 0.1 kHz'); sound(downsample(recOff1/max(abs(recOff1)), interpFactor), baseFs);
pause(maxLength/baseFs + 1);
disp('Exp B: LO Offset 1.0 kHz'); sound(downsample(recOff2/max(abs(recOff2)), interpFactor), baseFs);

figure('Name','Experiment B: LO Offset');
subplot(2,1,1); plotSpectrum(recOff1, newFs, 'Offset 0.1 kHz', 'r'); xlim([-bw*1.2, bw*1.2]);
subplot(2,1,2); plotSpectrum(recOff2, newFs, 'Offset 1.0 kHz', 'r'); xlim([-bw*1.2, bw*1.2]);
```

Note: Generates recovered spectra figures for both the 0.1 kHz and 1 kHz offsets.

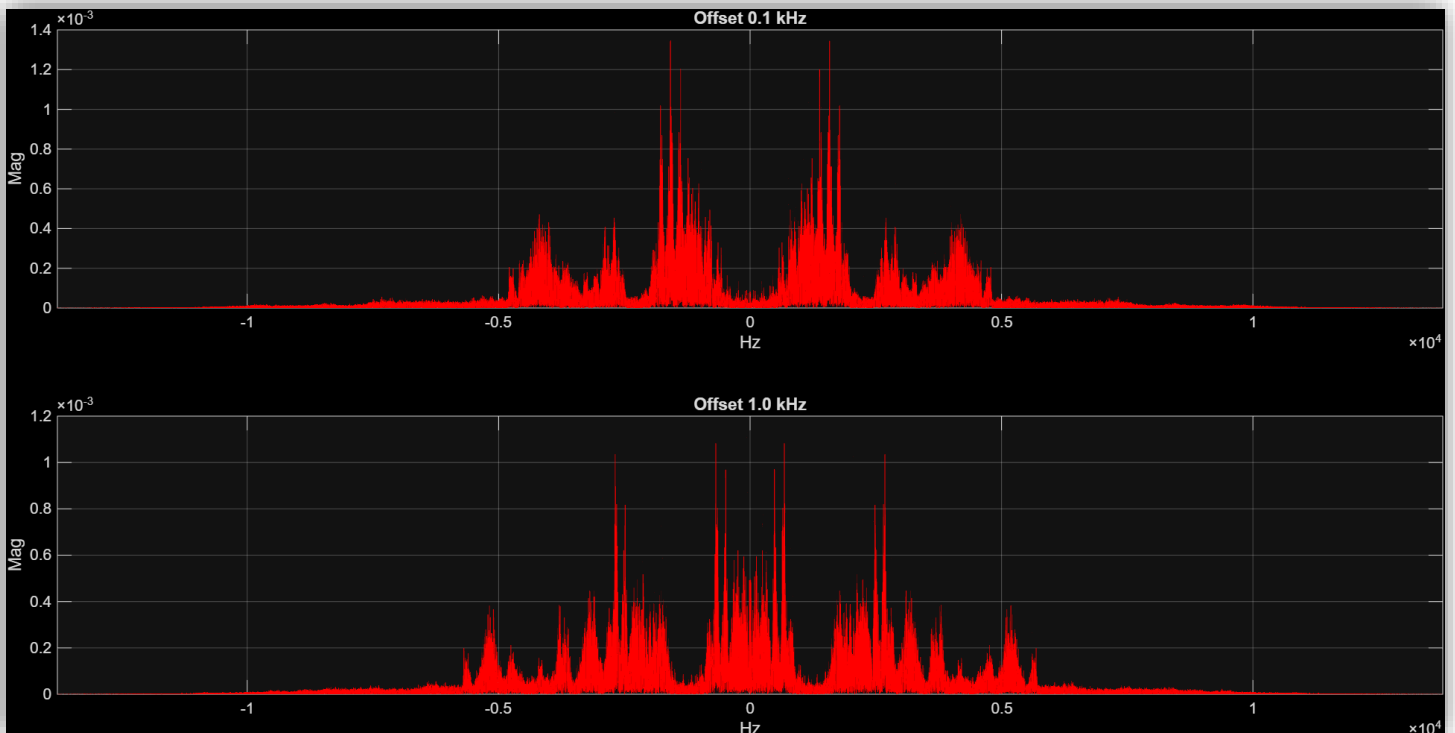


Figure 9: Recovered Spectra with 0.1 kHz and 1.0 kHz LO Offsets.

2.10 Helper Functions

Explanation:

plotSpectrum is used to calculate and visualize the FFT of signals, centering it around 0 Hz with fftshift.

estimateBandwidth calculates the frequency range where the signal magnitude exceeds 1% of its maximum, which informs filter design for the RF and IF stages.

MATLAB Code Snippet:

```
%% Helper Functions
function plotSpectrum(sig, fs, txt, clr)
    N = length(sig); f = (-N/2:N/2-1)*(fs/N);
    plot(f, abs(fftshift(fft(sig)))/N, 'Color', clr);
    grid on; title(txt); xlabel('Hz'); ylabel('Mag');
end

function bw = estimateBandwidth(sig, fs)
    N = length(sig); f = (-N/2:N/2-1)*(fs/N);
    mag = abs(fftshift(fft(sig))); mag = mag / max(mag);
    idx = find(mag > 0.01);
    if isempty(idx), bw = 5000; else, bw = max(abs(f(idx))); end
end
```

3. Questions & Answers

1. In two or three sentences, discuss the role of the RF, the IF and the baseband detector. Indicate why we need the IF stage?

The RF stage acts as a tunable band-pass filter to select the desired high-frequency station and perform interference-image rejection. The IF stage utilizes a fixed-frequency band-pass filter to cleanly isolate the signal with high selectivity, while the baseband detector shifts the IF signal to 0 Hz and applies a low-pass filter to retrieve the final message. The IF stage is essential because it is physically difficult and expensive to design a highly selective, sharp filter that can dynamically tune across a wide range of RF frequencies; shifting all targeted stations to a fixed IF allows the receiver to use one highly optimized, permanent filter.

2. Suppose you want to demodulate the first station (i.e. at ω_0), plot the spectrum of the outputs of the RF, the IF and the baseband stages.

- **RF Output:** The spectrum will show the signal cleanly isolated and centered at ± 100 kHz (the carrier frequency ω_0).

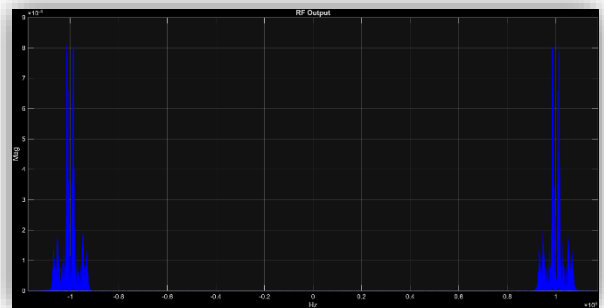


Figure 10: RF output spectrum of Signal 1.

- **IF Output:** The spectrum will shift down, perfectly centered at the fixed Intermediate Frequency of ± 15 kHz.

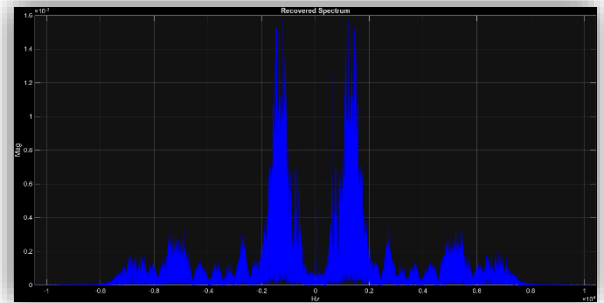


Figure 11: IF output spectrum of Signal 1.

- **Baseband Output:** The spectrum is finally down-mixed and centered at 0 Hz, representing the original audio message bandwidth (e.g., spanning roughly -12 kHz to 12 kHz).

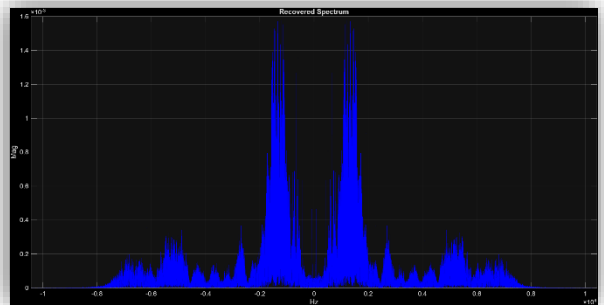


Figure 12: Recovered baseband spectrum of Signal 1.

3. Use the command 'sound' for the demodulated signal and check whether you can successfully listen to the radio station. Please comment on this step in your report.

Executing the sound command allows for clear, recognizable audio playback. This proves that the combination of upsampling, coherent demodulation, filtering, and subsequent downsampling successfully mathematically reversed the DSB-SC modulation process.

4. Repeat parts 2 and 3 but after removing the RF BPF. That is, the RF stage does not exist, what would happen if you try to demodulate the station at ω_0 ?

Removing the RF filter completely eliminates the receiver's image-rejection capabilities. If the FDM signal contains another broadcast at the "image frequency" (which sits exactly $2\omega_{IF}$ away from ω_0), the mixer will fold both the desired station and the image station down into the exact same 15 kHz IF band. The spectrum will show two overlapping signals, and the audio will sound like two scrambled tracks playing simultaneously.

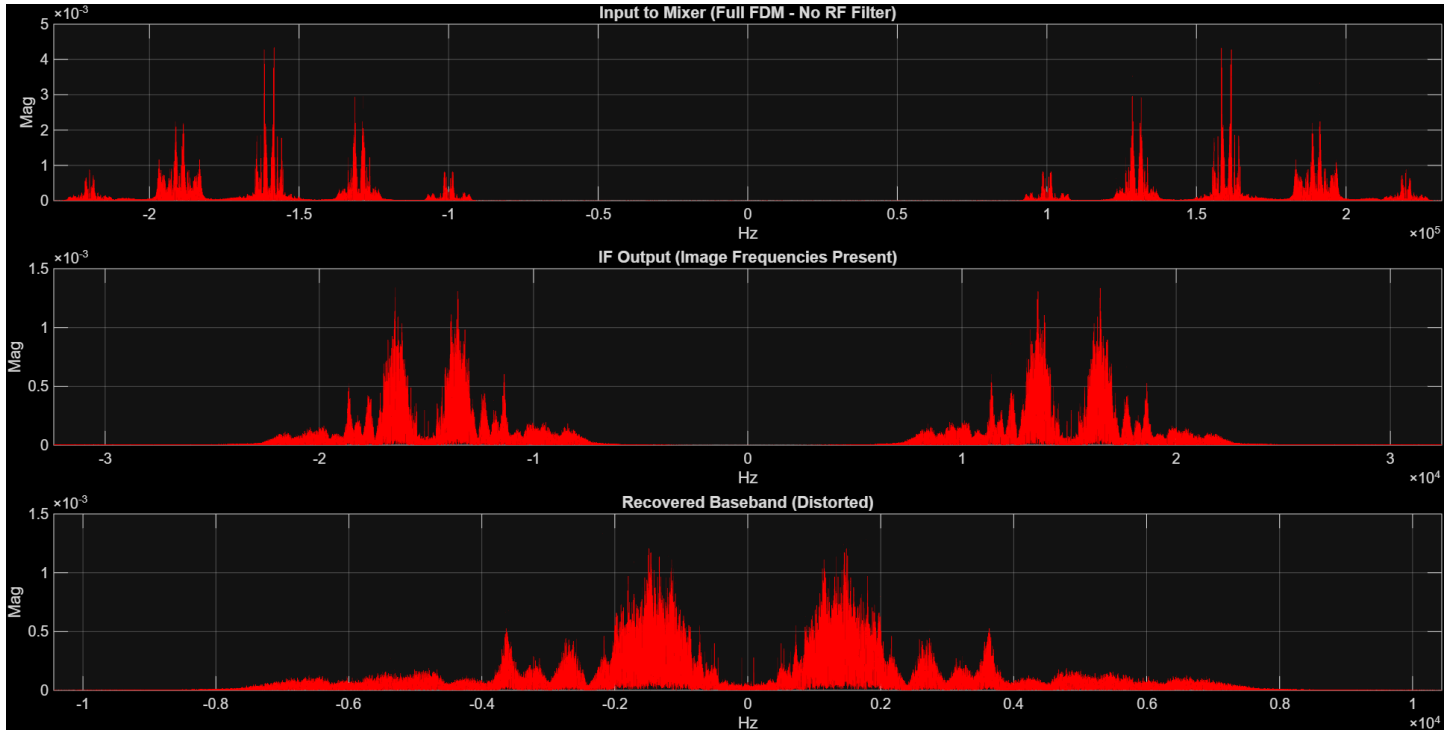


Figure 13: Effect of removing RF filter for Signal 1.

5. **What happens (in terms of the spectrum and the sound quality) if the receiver oscillator has frequency offset by 0.1 kHz and 1 kHz?**

- **0.1 kHz Offset:** The resulting baseband spectrum is inaccurately shifted by 100 Hz rather than perfectly landing at 0 Hz. The sound quality features a noticeable low-frequency tremolo or metallic pitch-shift effect, though it might still be partially intelligible.
- **1.0 kHz Offset:** The spectrum is drastically misaligned by 1000 Hz, causing severe signal distortion. The sound quality completely degrades into a "Donald Duck" effect, destroying the harmonic relationships in the voice and rendering the audio largely unintelligible.

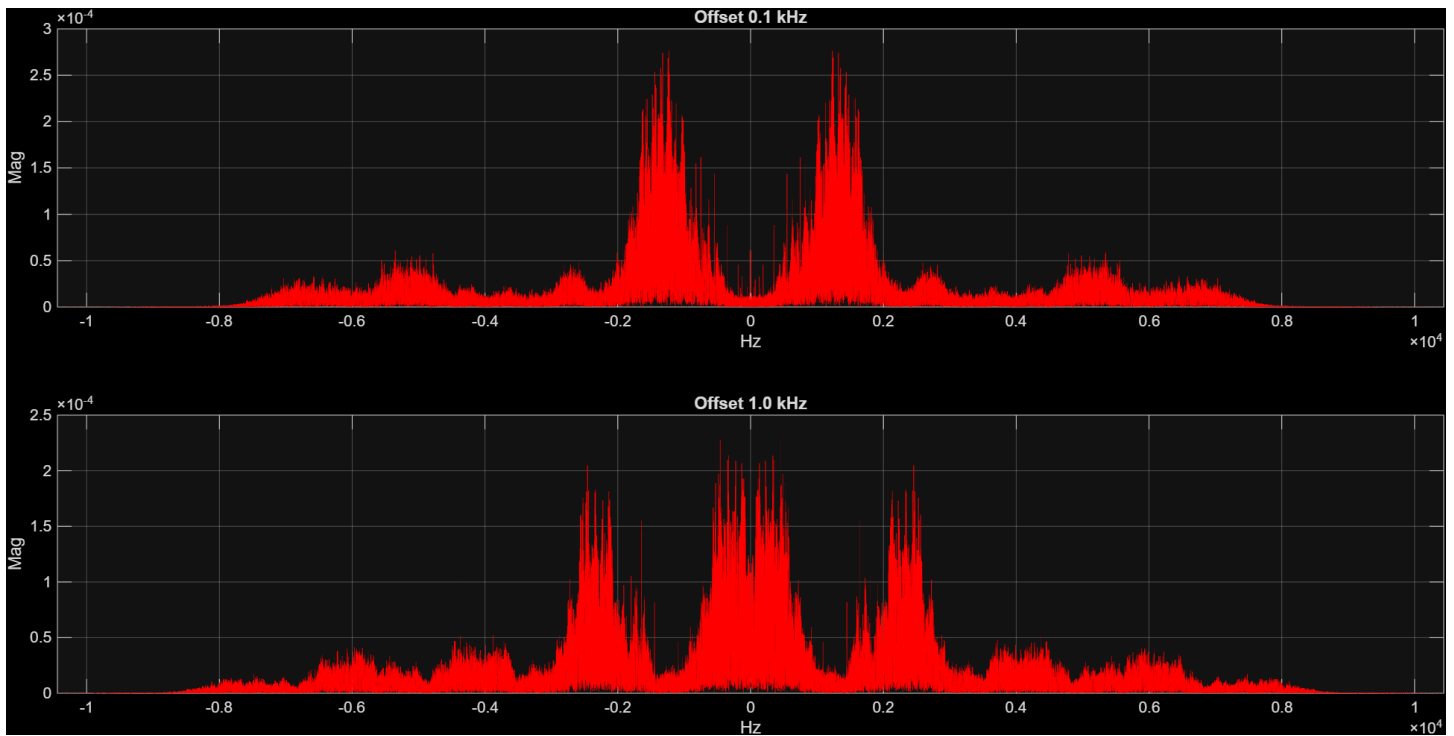


Figure 14: Recovered spectra with 0.1 kHz and 1.0 kHz LO offsets for Signal 1.

4. Conclusion

This project successfully modeled an analog super-heterodyne communication system utilizing Double Sideband Suppressed Carrier (DSB-SC) modulation to transmit and receive Frequency Division Multiplexed (FDM) audio channels. The simulation demonstrated the critical importance of the RF stage for image rejection, the IF stage for highly selective filtering, and the baseband detector for recovering the original message.

Through various experiments, the project also emphasized the system's strict dependency on precise local oscillator synchronization and filter design to avoid spectrum overlap, distortion, and degraded audio quality. Overall, the simulation provides both theoretical and practical insights into the design and operation of real-world AM radio receivers and illustrates the advantages of the super-heterodyne approach in managing multiple channels efficiently.

5. Full MATLAB Code and GitHub Link

5.1 Full MATLAB Code

```
% =====  
% Super-heterodyne Receiver & DSB-SC AM Communication System Simulation  
% =====  
clear; clc; close all;  
  
%% 1. Signal Preprocessing  
disp('--- Stage 1: Preprocessing ---');  
fileNames = {'audio1.wav', 'audio2.wav', 'audio3.wav', 'audio4.wav', 'audio5.wav'};  
numSig = length(fileNames); audioSignals = cell(1, numSig);  
fs_orig = zeros(1, numSig); maxLength = 0;  
  
for i = 1:numSig  
    [raw, fs] = audioread(fileNames{i});  
    fs_orig(i) = fs; mono = sum(raw, 2);  
    mono = mono / max(abs(mono)); audioSignals{i} = mono;  
    maxLength = max(maxLength, length(mono));  
end  
  
paddedSignals = zeros(maxLength, numSig);  
for i = 1:numSig, paddedSignals(1:length(audioSignals{i}), i) = audioSignals{i}; end  
  
%% 2. Interpolation & BW Estimation  
disp('--- Stage 2: Resampling & BW ---');  
baseFs = fs_orig(1); interpFactor = ceil(600000 / baseFs);  
newFs = baseFs * interpFactor; interpLen = maxLength * interpFactor;  
processedSignals = zeros(interpLen, numSig); estBW = zeros(1, numSig);  
  
for i = 1:numSig  
    estBW(i) = min(estimateBandwidth(paddedSignals(:,i), baseFs), 12000);  
    fprintf('Signal %d: BW = %.2f Hz\n', i, estBW(i));  
    processedSignals(:,i) = interp(paddedSignals(:,i), interpFactor);  
end  
  
%% 3. AM Modulation & Comparative Spectra  
disp('--- Stage 3: AM Modulation ---');  
t = (0:interpLen-1)' / newFs; modulatedSignals = zeros(interpLen, numSig);  
fc = 100000 + (0:numSig-1)*30000;  
figure('Name','Stage 3: Comparative Spectra'); colors = lines(numSig);  
  
for i = 1:numSig  
    modulatedSignals(:,i) = processedSignals(:,i) .* cos(2*pi*fc(i)*t);  
    subplot(2,1,1); hold on; plotSpectrum(processedSignals(:,i), newFs, '',  
    colors(i,:));  
    subplot(2,1,2); hold on; plotSpectrum(modulatedSignals(:,i), newFs, '',  
    colors(i,:));  
end  
subplot(2,1,1); title('All Baseband Spectra'); grid on; legend show; xlim([-  
max(estBW)*1.2, max(estBW)*1.2]);  
subplot(2,1,2); title('All Modulated Spectra'); grid on; legend show; xlim([-  
(max(fc)+max(estBW)), max(fc)+max(estBW)]);  
  
%% 4. Frequency Division Multiplexing (FDM)  
disp('--- Stage 4: FDM ---');  
fdmSignal = sum(modulatedSignals, 2);
```

```

figure('Name','Stage 4: FDM'); plotSpectrum(fdmSignal, newFs, 'FDM Spectrum', 'b');
xlim([- (max(fc)+max(estBW)), max(fc)+max(estBW)]);

%% 5. Receiver - RF Stage
disp('--- Stage 5: RF Stage ---');
targetIdx = 3; targetFc = fc(targetIdx); bw = estBW(targetIdx);
rfFilt = designfilt('bandpassiir','FilterOrder',4,'HalfPowerFrequency1',targetFc-bw,...
    'HalfPowerFrequency2',targetFc+bw,'SampleRate',newFs);
rfOut = filtfilt(rfFilt, fdmSignal);
figure('Name','Stage 5: RF Output'); plotSpectrum(rfOut, newFs, 'RF Output', 'b');
xlim([- (targetFc + 2*bw), targetFc + 2*bw]);

%% 6. Mixer & Local Oscillator
disp('--- Stage 6: Mixer ---');
fIF = 15000; fLO = targetFc + fIF;
mixedSignal = rfOut .* cos(2*pi*fLO*t);
figure('Name','Stage 6: Mixer'); plotSpectrum(mixedSignal, newFs, 'Mixer Output', 'b');
xlim([- (fLO + targetFc + bw), (fLO + targetFc + bw)]);

%% 7. IF Stage
disp('--- Stage 7: IF Stage ---');
ifFilt = designfilt('bandpassiir','FilterOrder',4,'HalfPowerFrequency1',fIF-bw,...
    'HalfPowerFrequency2',fIF+bw,'SampleRate',newFs);
ifOut = filtfilt(ifFilt, mixedSignal);
figure('Name','Stage 7: IF Output'); plotSpectrum(ifOut, newFs, 'IF Output', 'b');
xlim([- (fIF + 2*bw), fIF + 2*bw]);

%% 8. Baseband Detection & Playback
disp('--- Stage 8: Demodulation ---');
detMixed = ifOut .* cos(2*pi*fIF*t);
lpFilt =
designfilt('lowpassiir','FilterOrder',6,'HalfPowerFrequency',bw,'SampleRate',newFs);
recovered = filtfilt(lpFilt, detMixed); recovered = recovered / max(abs(recovered));

figure('Name','Stage 8: Recovered'); plotSpectrum(recovered, newFs, 'Recovered
Spectrum', 'b');
xlim([-bw*1.2, bw*1.2]);
disp('Playing recovered audio...'); sound(downsample(recovered, interpFactor), baseFs);
pause(maxLength/baseFs + 1);

%% 9. Experiments
disp('--- Stage 9: Experiments ---');
% Exp A: No RF Filter
disp('Exp A: No RF Filter (Playing audio...)');
mixedNoRF = fdmSignal .* cos(2*pi*fLO*t);
ifOutNoRF = filtfilt(ifFilt, mixedNoRF);
recNoRF = filtfilt(lpFilt, ifOutNoRF .* cos(2*pi*fIF*t));

figure('Name','Experiment A: Effect of Removing RF Filter');
subplot(3,1,1); plotSpectrum(fdmSignal, newFs, 'Input to Mixer (Full FDM - No RF
Filter)', 'r');
xlim([- (max(fc)+max(estBW)), max(fc)+max(estBW)]);
subplot(3,1,2); plotSpectrum(ifOutNoRF, newFs, 'IF Output (Image Frequencies Present)',
'r');
xlim([- (fIF + 2*bw), fIF + 2*bw]);
subplot(3,1,3); plotSpectrum(recNoRF, newFs, 'Recovered Baseband (Distorted)', 'r');
xlim([-bw*1.2, bw*1.2]);

```

```

sound(downsample(recNoRF/max(abs(recNoRF)), interpFactor), baseFs);
pause(maxLength/baseFs + 1);

% Exp B: LO Offset
off1 = 100; off2 = 1000;
recOff1 = filtfilt(lpFilt, filtfilt(ifFilt, rfOut .* cos(2*pi*(fLO+off1)*t)) .*
cos(2*pi*fIF*t));
recOff2 = filtfilt(lpFilt, filtfilt(ifFilt, rfOut .* cos(2*pi*(fLO+off2)*t)) .*
cos(2*pi*fIF*t));

disp('Exp B: LO Offset 0.1 kHz'); sound(downsample(recOff1/max(abs(recOff1)),
interpFactor), baseFs);
pause(maxLength/baseFs + 1);
disp('Exp B: LO Offset 1.0 kHz'); sound(downsample(recOff2/max(abs(recOff2)),
interpFactor), baseFs);

figure('Name','Experiment B: LO Offset');
subplot(2,1,1); plotSpectrum(recOff1, newFs, 'Offset 0.1 kHz', 'r'); xlim([-bw*1.2,
bw*1.2]);
subplot(2,1,2); plotSpectrum(recOff2, newFs, 'Offset 1.0 kHz', 'r'); xlim([-bw*1.2,
bw*1.2]);

%% Helper Functions
function plotSpectrum(sig, fs, txt, clr)
    N = length(sig); f = (-N/2:N/2-1)*(fs/N);
    plot(f, abs(fftshift(fft(sig)))/N, 'Color', clr);
    grid on; title(txt); xlabel('Hz'); ylabel('Mag');
end

function bw = estimateBandwidth(sig, fs)
    N = length(sig); f = (-N/2:N/2-1)*(fs/N);
    mag = abs(fftshift(fft(sig))); mag = mag / max(mag);
    idx = find(mag > 0.01);
    if isempty(idx), bw = 5000; else, bw = max(abs(f(idx))); end
end

```

2.2 GitHub Repository

<https://github.com/Mohaned-Waleed-Hosny/Superheterodyne-AM-Receiver>